

```

{
//write all available port names
portNames.Append(i+" "+portname + " ");
i++;
}

/* Updating the UI TextBlock for COM
 * port names */
sysPortName.Text = portNames.
ToString();
}

```

With the help of the above code we will get all COM ports/serial ports available in the system. After we get the names, we will need to display these to the user to specify which COM ports correspond to the RFID reader. If we get the port name "COM1," "COM2," it will be displayed to the user as "1)COM1 2)COM2." Likewise, the user must select either the values 1 or 2 to select the desired COM port. The serial port code is as listed below:

```

/* When any text is written/deleted
in the portNum TextBox in the UI
 * then this TextChanged event handler
is called */
private void portNum_TextChanged(object
sender, RoutedEventArgs e)
{
// Check if PortState is true or false
if (!portstate)
{
try
{
int portNumber = Convert.
ToInt32(portNum.Text);
//create serial port as configured below
//BaudRate = 9600 bps, Parity:None,
8-bit data, 1 Stop-bit
port1 = new SerialPort(ports[portNumber
- 1], 9600, Parity.None, 8, StopBits.
One);

//create asynchronous event(it's a
secondary thread of the main program)
//will activate every time our serial
device sends data to computer
port1.DataReceived += new SerialDataRe
ceivedEventHandler(port1_DataReceived);
//finally open port
port1.Open();
portstate = true;
this.portStatus.Text = "Port " + port
Number + " is open !!";
}
}
}

```

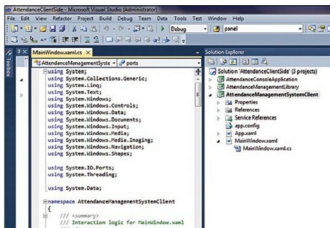


Fig. 24: MainWindow.xaml.cs

```

}
catch (Exception)
{
this.portStatus.Text = "Port can't
be opened !!";
}
}
else
{
this.portStatus.Text = "A port is
already open !!";
}
}
}

```

There is an event handler in MainWindow.xaml.cs that is called whenever there is a text change in portNum TextBox in the UI. This is mainly to take the choice of the COM port from the user. Here, the chosen COM port is opened using the following parameters.

```

Baudrate: 9600
Parity: None
Data bits: 8
Stop bits: 1

```

Once the user has entered the desired port and the port has successfully opened, the user can no longer change the choice. In case the user needs to do so, he or she will need to close the application and re-open it to change the COM port. This restriction has been taken care of by flag portstate.

If a COM port is successfully opened, the event handler is called when data is received in the port. The string of data that is received is stored in the global variable data.

We now create an object of our service reference. We can pass the name of the endpoint in ServiceReference.ServiceClient constructor to uniquely identify our endpoint to be used in the client. This can be retrieved from app.config of our client project. Here, it is BasicHttpBinding_

ing_IAttendanceRegisterService.

Note. We made a simple logic such that log-in and log-out times are updated alternately, each time the same RFID tag is in the proximity of the RFID reader. So, if loginCount is even, we consider that it can be LoginTime, and if it is odd, it can be LogoutTime. The logic can be further enhanced based on user requirements.

To keep track of log-in and log-out times of employees, we have created database update logic in port1_DataReceived event handler in MainWindow.xaml.cs. We read the port data and write it in a class-level variable, which through a different thread is updated in the UI thread. So, our client application should be running all the time. Every time the employee brings the RFID tag near the RFID reader, the timing is updated in the database. If at a later point in time the admin wants to check the timings, they can use Employee Data tab in the WPF UI and get the information.

The function of the thread given below is to check whether any new RFID data is received and displayed in the UI. As we have already mentioned before, the received data is stored in the global variable data; any change in the variable value is scanned by this thread and displayed in the UI.

```

/* The method that runs inside the newly
created thread to update UI */
public void MyThread()
{
// Old Rfid data

```